```
function fNN(node, location, predicate, (nnDist, nn)=(inf,
null))    #1
  if node == null then
                                                        #2
    return (nnDist, nn)

else
     #3
    dist ← distance(node.shop.point,
location)                                  #4
    if predicate(node) and dist < nnDist
then                                      #5
      (nnDist, nn) ← (dist, node.shop)
    if compare(location, node) < 0
then                                        #6
      closeBranch ← node.left
      farBranch ← node.right
    else
                #7
      closeBranch ← node.right
      farBranch ← node.left
    (nnDist, nn) ← fNN(closeBranch, location, predicate,
(nnDist, nn)) #8
    if splitDistance(location, node) < nnDist
then                    #9
      (nnDist, nn) ← fNN(farBranch, location, predicate,
(nnDist, nn))        #10
    return (nnDist,
nn)                                                          #11
```

#1 Find the closest point to a given target, among those satisfying a predicate. We also pass best the values found so far for nearest neighbor (*NN*) and its distance, to help pruning. These values default to `null`, `infinity` for a call on the tree root.